MINIOS

User Manual

A practical guide to logging in, navigating the terminal, managing files, and using simulated memory and process commands.

1. Introduction

MiniOS Alpha is a Python-based command-line operating system simulator. It demonstrates familiar operating-system concepts through a text terminal: user login, a file and directory manager, simulated RAM allocation, and a simple process table. Users type commands and read the results directly in the terminal.

> **Please note:** MiniOS is a learning simulator, not a replacement for Windows, macOS, or Linux. Its memory and process features are simulated; they do not allocate actual RAM or launch real applications.

2. System Requirements

- Python 3 installed on the computer. The program uses Python standard-library modules only; no third-party packages are required.
- A command-line terminal: Command Prompt or PowerShell on Windows, or Terminal on macOS/Linux.
- The MiniOS Python script saved in a folder where the user has permission to create and update files.
- Enough disk space for the script and the text files stored in the MiniOS_Disk folder.

> **Storage behavior:** By default, MiniOS keeps its managed files in a folder named MiniOS_Disk beside the Python script. Keep unrelated files out of this folder because MiniOS synchronizes the folder to match its virtual file tree.

3. Installation

MiniOS is a standalone Python script. Use the following steps to set it up:

- Save the provided source code in a file, for example, minios.py.
- Install Python 3 if it is not already installed. On Windows, enable the option to add Python to PATH if the installer offers it.
- Place minios.py in a normal working folder where you have write permission.
- Open a terminal and change to that folder. For example, use cd followed by the folder path.
- Start MiniOS using the command shown in Section 4.

> **First launch:** MiniOS automatically creates MiniOS_Disk next to the script if that folder does not already exist. If the folder already contains files or directories, MiniOS loads them into its virtual file tree.

4. How to Start MiniOS

Open a terminal in the folder containing the Python script, then run the command that matches your Python installation:

```
python minios.py
```

On some macOS or Linux installations, the command is:

```
python3 minios.py
```

Replace minios.py with the actual name of your saved script if it is different. The program displays the MiniOS LOGIN screen and requests credentials. After a successful login, it displays “MiniOS Alpha” and a prompt such as MiniOS:/>. Type a command after the prompt and press Enter.

5. Login Instructions

The program has two built-in accounts. Enter one of the following username and password pairs when prompted:

Username	Password
admin	admin123
student	student123

The password is entered using a hidden-input prompt, so the characters may not appear as you type. If the login fails, check the spelling and try again. Pressing Ctrl+C or sending end-of-input cancels login.

> **Security reminder:** These are demonstration credentials stored directly in the source code. Do not use this login system to protect sensitive information.

6. Command List

Commands are entered at the MiniOS prompt. Command names are not case-sensitive. Use the syntax shown for each command. In the examples below, words in angle brackets such as <file> represent values you supply; do not type the angle brackets themselves. Use quotation marks around a path or filename that contains spaces.

6.1 General and system commands

help

Purpose: Shows the registered MiniOS commands and the file-management command list.
Syntax: `help`
Example: `help`

clear

Purpose: Clears the terminal display.
Syntax: `clear`
Example: `clear`

date

Purpose: Displays the computer’s current date in YYYY-MM-DD format.
Syntax: `date`
Example: `date`

time

Purpose: Displays the computer’s current time in 24-hour HH:MM:SS format.
Syntax: `time`
Example: `time`

whoami

Purpose: Displays the username of the currently logged-in MiniOS user.
Syntax: `whoami`
Example: `whoami`

systeminfo

Purpose: Displays simulated MiniOS version, architecture, virtual CPU, memory, and current user details.
Syntax: `systeminfo`
Example: `systeminfo`

exit
Purpose: Ends the MiniOS command loop and displays the shutdown message.
Syntax: `exit`
Example: `exit`

6.2 File and directory commands

dir
Purpose: Lists files and folders in the current directory or a specified path.
Syntax: `dir [path]`
Example: `dir Documents`

ls
Purpose: Alias for dir. Lists files and folders.
Syntax: `ls [path]`
Example: `ls`

mkdir
Purpose: Creates one or more new directories.
Syntax: `mkdir <folder> [folder2 ...]`
Example: `mkdir Documents`

create
Purpose: Creates a new text file, optionally with starting text. The file must not already exist.
Syntax: `create <file> [text]`
Example: `create notes.txt First draft`

read
Purpose: Displays the contents of a text file.
Syntax: `read <file>`
Example: `read notes.txt`

type
Purpose: Alias for read. Displays the contents of a text file.
Syntax: `type <file>`
Example: `type notes.txt`

cat
Purpose: Alias for read. Displays the contents of a text file.
Syntax: `cat <file>`
Example: `cat notes.txt`

write
Purpose: Replaces all existing contents of a text file with the supplied text.
Syntax: `write <file> <text>`
Example: `write notes.txt Final draft`

append
Purpose: Adds supplied text to the end of an existing text file. It does not insert a line break automatically.
Syntax: `append <file> <text>`
Example: `append notes.txt Additional notes`

delete
Purpose: Deletes a file, or deletes a directory if it is empty. Add /s to delete a directory and its contents.
Syntax: `delete <path> [/s]`
Example: `delete OldNotes /s`

del

Purpose: Alias for delete.
Syntax: `del <path> [/s]`
Example: `del notes.txt`

rename

Purpose: Changes the name of an existing file or directory. Supply only the new name as the second argument, not a full destination path.
Syntax: `rename <path> <newname>`
Example: `rename notes.txt class-notes.txt`

cd

Purpose: Changes the current directory. With no path, returns to the root directory. Use .. to move up one level.
Syntax: `cd [path]`
Example: `cd Documents`

pwd

Purpose: Displays the full path of the current directory.
Syntax: `pwd`
Example: `pwd`

copy

Purpose: Copies a file or directory to a destination. If the destination is an existing directory, the source name is kept inside it.
Syntax: `copy <source> <destination>`
Example: `copy notes.txt backup.txt`

move

Purpose: Moves a file or directory to a destination. If the destination is an existing directory, the source name is kept inside it.
Syntax: `move <source> <destination>`
Example: `move notes.txt Documents`

tree

Purpose: Displays the directory tree starting at the current directory or a specified directory.
Syntax: `tree [path]`
Example: `tree Documents`

fhelp

Purpose: Displays help for file and directory commands.
Syntax: `fhelp`
Example: `fhelp`

> **File safety:** The copy and move commands do not overwrite an existing destination item. To delete the current directory or one of its parent directories, first use cd to move somewhere else. The root directory cannot be deleted or renamed.

6.3 Memory-management commands

MiniOS models a total of 1,024 MB of memory, with 256 MB reserved for the system. This leaves 768 MB initially available for user allocations. The values change only inside the simulation and reset when MiniOS restarts.

memory

Purpose: Shows total, used, and free simulated memory in MB.
Syntax: `memory`
Example: `memory`

allocate

Purpose: Reserves a positive whole number of megabytes from the simulated free memory pool.
Syntax: `allocate <MB>`
Example: `allocate 100`

free

Purpose: Releases a positive whole number of megabytes previously allocated by the user. It cannot release the 256 MB reserved for the system.
Syntax: `free <MB>`
Example: `free 50`

> **Example:** After allocate 100, the simulated user allocation is 100 MB. Running free 50 reduces the user allocation to 50 MB.

6.4 Process-management commands

The process table starts with two protected entries: PID 1 (system_idle) and PID 2 (shell). New simulated processes receive increasing process IDs, starting at 3.

process

Purpose: Lists the current simulated processes, their PIDs, names, and statuses.
Syntax: `process`
Example: `process`

run

Purpose: Adds a named process to the simulated process table. It does not launch an actual computer application.
Syntax: `run <name>`
Example: `run Calculator`

kill

Purpose: Removes a simulated process from the table using its numeric PID. The two core system processes cannot be terminated.
Syntax: `kill <PID>`
Example: `kill 3`

6.5 Tips for entering commands

- Press Enter after typing each command.
- Use dir or tree to check names and paths before editing or deleting items.
- Use quotation marks around names containing spaces, for example: cd "Class Files".
- Use help for the full command overview and fhelp for file-management help.
- MiniOS files are treated as text. It is not intended for storing binary files such as images, audio, or executable programs.

7. Common Errors

MiniOS usually prints an error message when a command cannot be completed. Use this table to identify the cause and the usual fix.

Message / situation	Cause and what to do
Invalid username or password	Check that you entered exactly admin/admin123 or student/student123. Password entry is hidden.
Command is not recognized	Check the spelling. Type help to see supported commands.
Missing argument / Usage: ...	The command needs more or fewer arguments. Repeat it using the syntax in Section 6.
... was not found	The file or directory does not exist at the path you entered. Run dir or tree and check the path spelling.
Already exists	An item with that name is already in the destination. Choose a different name or destination.
Not a directory	A path component is a file rather than a folder. Check the path and use cd only with directories.
Is a directory, not a file	A file-only command was used on a directory. Use dir or tree to inspect folders.
Directory is not empty	Use delete <directory> /s only if you intend to delete the directory and all its contents.
Cannot delete the current directory (or one that contains it)	Use cd to leave that directory and its descendants first, then retry.
Cannot delete / rename / copy / move the root directory	The root is protected by the simulator. Choose a file or subdirectory instead.
Cannot copy/move a directory into itself	Choose a destination outside the source directory and its descendants.
Not enough memory	The requested amount exceeds the current simulated free memory. Run memory and request a smaller amount.
Cannot free ... Only ... user-allocated memory is in use	You tried to release more memory than was allocated. Check memory and free only the user-allocated amount.
Amount must be greater than 0 / invalid whole number	Use a positive integer number of megabytes, such as allocate 64.
Invalid PID format / Process does not exist	Use process to find a current PID, then enter that numeric PID with kill.
Cannot terminate core system process	PID 1 (system_idle) and PID 2 (shell) are protected and cannot be terminated.
MiniOS_Disk write or permission problem	Make sure the script folder and MiniOS_Disk are writable. Close or remove any conflicting file/folder only after checking it is safe to do so.

Important data warning: Every file-changing command synchronizes the MiniOS_Disk folder to match MiniOS's virtual tree. Items in that folder that are not in the virtual tree may be removed during synchronization. Keep backups of important files and do not use MiniOS_Disk for unrelated data.

8. How to Exit MiniOS

At the MiniOS prompt, type the following command and press Enter:

```
exit
```

MiniOS displays "Shutting down MiniOS... Goodbye!" and returns control to the terminal. You can also press Ctrl+C or send end-of-input to interrupt the session, but using exit is the normal way to shut down.

Before exiting: Make sure any important text has been saved with write or append. File and directory changes are synchronized to MiniOS_Disk when each mutating file command completes; simulated memory allocations and processes are not preserved after restart.